

Memoria Práctica 1: Aritmética Modular

Miembros del grupo:

Miguel Gabaldón Poncela

Gonzalo García Martínez-Echevarría

GP10A

Diseño general del programa:

Se ha desarrollado el programa utilizando una estructura modular de programación con distintas funciones. No se han implementado librerías adicionales propias pero si que se han utilizado algunas externas como `math` o `re`. El uso de estas librerías se explicará más en detalle en cada una de las funciones usadas. De la misma manera las funciones también se irán explicando con más detalle.

Modular:

Es_primo()

El algoritmo implementado se basa en la **Criba de Eratóstenes** optimizada para la comprobación de primalidad, pero adaptada para comprobar la divisibilidad solo hasta la raíz cuadrada de n . Los pasos principales son:

1. **Casos triviales:** Si n es menor o igual a 1, se devuelve `False` ya que no son primos. Si n es 2 o 3, se devuelve `True` ya que ambos son primos. Esto se hace para evitar posibles errores iniciales
2. **Divisibilidad por 2 y 3:** Si n es divisible por 2 o 3, se devuelve `False`. Ya que así nos vamos quitando una gran cantidad de números que puede que sean primos
3. **Iteración optimizada:** A partir de aquí, se itera desde $i=5$ incrementando en pasos de 6 (es decir, 5, 7, 11, 13, ...) para verificar si n es divisible por i o $i+2$. Esto reduce la cantidad de divisiones necesarias, ya que se ignoran números pares y múltiplos de 3.
4. **Límite de la raíz cuadrada:** La comprobación solo se realiza hasta la raíz cuadrada de n ya que, si un número es mayor a la raíz cuadrada y divide a n , entonces un cociente menor ya lo habría hecho. Se acota la búsqueda de divisores. Si se busca una mayor explicación de este concepto se puede encontrar en el anexo (Tma de referencia 1)

Lista_primos()

El algoritmo implementado se fundamenta en el método `es_primo()` realizado previamente. En este se va iterando dentro del intervalo dado y se va llamando a la función `es_primo()`. Se va añadiendo a la lista aquellos que sean primos. La fundamentación matemática ya está explicada en la función anterior.

factorizar()

La función factorizar descompone un número n en sus factores primos y los almacena en un diccionario donde las claves son los primos y los valores son sus exponentes.

1. **Manejo de números negativos:** Si n es negativo, se convierte en positivo, para el funcionamiento del programa ya que solo se piden los factores positivos. Además, si se factoriza un número en primos no pueden aparecer números negativos puesto que estos nunca serán primos.
2. **Divisibilidad por 2:** Se procesa el factor 2 primero, dividiendo n hasta que ya no sea par, y se actualiza el diccionario en consecuencia.
3. **Búsqueda de divisores impares:** Se busca la divisibilidad por números impares, desde 3 hasta la raíz cuadrada de n , para reducir el número de comprobaciones.
4. **Comprobación final:** Si al final n es mayor que 1, significa que n es un número primo mayor que su propia raíz cuadrada, por lo que se añade al diccionario.

El fundamento matemático detrás del algoritmo se basa en el **Teorema Fundamental de la Aritmética**, que establece que todo número entero positivo mayor que 1 se puede descomponer de manera única en un producto de números primos elevados a sus respectivos exponentes:

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k} = \prod_{i=1}^k p_i^{\alpha_i}$$

Además, solo es necesario buscar divisores hasta la raíz cuadrada de n porque si n tuviera un divisor mayor que su raíz cuadrada, el otro factor correspondiente sería menor que la raíz cuadrada, y ya se habría encontrado en pasos anteriores(`es_primo()`).

mcd()

La función mcd implementa el **Algoritmo de Euclides** para calcular el máximo común divisor (MCD) de dos números enteros a y b . Este algoritmo se basa en la propiedad de que el MCD de dos números no cambia si el número mayor es reemplazado por su diferencia con el menor. Sin embargo, en esta implementación se utiliza el **resto** de la división en lugar de la diferencia, lo cual es más eficiente.

1. **Bucle iterativo:** Mientras b no sea cero, se actualizan a y b de la siguiente forma:
 - a toma el valor de b .
 - b toma el valor de $a \% b$ (el residuo de la división de a entre b).
2. **Retorno:** Cuando b es igual a cero, se retorna el valor de a , que en este punto es el MCD de los números iniciales.

Justificación matemática

El **Algoritmo de Euclides** se fundamenta en la siguiente propiedad:

- Si d divide a a y a b, entonces también divide a a-b.

$$\text{mcd}(a, b) = \text{mcd}(b, a \% b)$$

Esta relación se aplica hasta que uno de los números sea cero, momento en el que el otro número es el MCD. Esta propiedad se puede sacar de las diapositivas aportadas de clase.

Bezout()

La función Bezout implementa el **Algoritmo Extendido de Euclides** para calcular el máximo común divisor (MCD) de dos números a y b, así como los coeficientes x e y que satisfacen la **Identidad de Bézout**:

Teorema de Bezout

Sean $a, b \in \mathbb{Z}$. Entonces existen $x, y \in \mathbb{Z}$ tales que:

$$xa + yb = \text{mcd}(a, b)$$

Caso base: Si $b=0$ o $a=0$, el MCD será el otro número aportado y los coeficientes son 1 y 0 dependiendo de cual sea 0.

Se realiza un algoritmo recursivo donde si no se ha hallado la solución se llama a `bezout(b, a % b)` para calcular el MCD y los coeficientes de la subecuación.

Actualización de coeficientes: Los coeficientes x y y se calculan en base a los valores devueltos en la recursión, usando la relación:

$$x = y1$$

$$y = x1 - \left(\frac{a}{b}\right) \times y1$$

La anterior actualización de coeficientes se basa en el teorema de bezout y el algoritmo de Euclides extendido vistos en clase.

1. **Recursión eficiente:** La función usa una estructura recursiva simple que aprovecha la reducción de los números a y b mediante el módulo, lo que garantiza que el algoritmo converge rápidamente a $b=0$.

coprimos()

La función `coprimos` determina si dos números enteros a y b son coprimos. Esto se logra calculando el máximo común divisor (MCD) de a y b utilizando la función `mcd`, explicada anteriormente.

Dos números a y b son **coprimos** (o **primos entre sí**) si y solo si su MCD es 1. Matemáticamente, esto se puede expresar como:

$$\text{mcd}(a, b) = 1 \implies a \text{ y } b \text{ son coprimos}$$

La explicación y fundamentos matemáticos de esta función se encuentran en la del MCD.

Potencia_mod_p()

La función `potencia_mod_p` implementa el **Algoritmo de Exponenciación Binaria**, que calcula eficientemente la potencia de una base elevada a un exponente bajo un módulo p . El algoritmo utiliza un bucle que divide el exponente en cada iteración. Si el exponente es impar, multiplica el resultado por la base. Luego, siempre eleva la base al cuadrado y reduce nuevamente módulo p . El bucle se ejecuta hasta que el exponente es cero. El resultado final es la potencia calculada módulo p .

Justificación matemática

El algoritmo se basa en la propiedad de que cualquier exponente puede ser descompuesto en potencias de 2:

$$\text{base}^{\text{exp}} = \text{base}^{b_0} \times (\text{base}^2)^{b_1} \times (\text{base}^4)^{b_2} \times \dots$$

Donde $\text{exp} = b_0 + 2b_1 + 4b_2 + \dots$ es la representación binaria del exponente.

inversa_mod_p()

La función `inversa_mod_p` encuentra el inverso multiplicativo de un número n bajo un módulo p . Se basa en el **Algoritmo Extendido de Euclides** para calcular el inverso, garantizando la siguiente expresión.

$$n \times \text{inverso} \equiv 1 \pmod{p}.$$

1. **Normalización de n :** Si n es negativo, se ajusta para ser positivo, añadiendo el módulo tantas veces como sea necesario.
2. **Verificación de coprimalidad:** Antes de proceder, se comprueba si n y p son coprimos usando la función `coprimos`. Si no lo son no existe el inverso.

Se implementa un bucle para actualizar coeficientes hasta encontrar el inverso. Los coeficientes se ajustan en cada iteración utilizando el cociente de la división de x por y .

La base teórica de esta función se encuentra en el **Teorema de Bézout**. Ya explicado en la misma función de `Bezout`. El algoritmo extiende esta relación para encontrar un x que sea el inverso multiplicativo de n bajo módulo p . La solución se obtiene cuando el máximo común divisor (MCD) entre n y p es 1, permitiendo reorganizar la ecuación para resolver el inverso.

euler()

La función euler calcula el valor de la **Función Phi de Euler** para un entero positivo n. La función phi cuenta cuántos enteros positivos menores que n son coprimos con n. Los pasos principales son:

1. **Reducción inicial de factores pares:** Si n es par, la función elimina todas las potencias de 2, ajustando el valor de resultado en consecuencia.
2. **Reducción de factores impares:** Después de manejar los factores pares, la función recorre los números impares desde 3 hasta la raíz cuadrada de n. Para cada factor primo que divide a n, se reduce resultado.

La **Función Phi de Euler** se define como:

$$\varphi(n) = \# \{k \in \mathbb{N} \mid k \leq n, (k, n) = 1\}$$

El algoritmo se asegura de que, al iterar sobre todos los factores primos y reducir n, se ajusta el valor de $\phi(n)$ correctamente sin recalcular divisiones innecesarias. Al iterar solo hasta la raíz cuadrada de n y manejar el caso cuando n es mayor que 1 al final, el algoritmo optimiza su eficiencia.

legendre()

La función legendre calcula el Símbolo de Legendre de un número n con respecto a un número primo p. Este símbolo, indica si n es un cuadrado perfecto módulo p. Los pasos principales son:

1. Verificación del módulo: Si el módulo =0, se lanza una excepción de división por cero, ya que no es válido calcular el símbolo de Legendre con un módulo cero.
2. Comprobación de divisibilidad: Si n es divisible por p, el símbolo de Legendre es 0.
3. Cálculo del símbolo de Legendre:
 - Se eleva n a la potencia $((p-1)/2)$ módulo p utilizando la función potencia_mod_p, mencionada anteriormente.
 - Si el resultado es p-1, el símbolo de Legendre es -1.
 - Si el resultado es 1, el símbolo de Legendre es 1.

Justificación matemática

El Símbolo de Legendre se define para un número entero n y un número primo impar p de la siguiente manera:

$$\left(\frac{n}{p}\right) = \begin{cases} 0 & \text{si } n \equiv 0 \pmod{p} \\ 1 & \text{si existe un entero } x \text{ tal que } x^2 \equiv n \pmod{p} \\ -1 & \text{en caso contrario} \end{cases}$$

La función utiliza esta definición y el Criterio de Euler, que establece que:

$$\left(\frac{n}{p}\right) \equiv n^{\frac{p-1}{2}} \pmod{p}$$

Si $n^{\frac{p-1}{2}} \equiv 1 \pmod{p}$ entonces n es un residuo cuadrático y el símbolo de Legendre es 1. Si sin embargo es congruente con $-1 \pmod{p}$, entonces n no es un residuo cuadrático y el símbolo de Legendre será -1 .

resolver_sistema_congruencia()

La función `resolver_sistema_congruencias` resuelve un sistema de congruencias lineales utilizando el **Teorema Chino del Resto (CRT)** y la **Exponenciación Modular** para encontrar el inverso modular.

1. **Inicialización:** La función comienza con $r \equiv y \pmod{1}$, donde r será el valor de la solución y mod el módulo acumulado.
2. **Iteración sobre cada congruencia:**
 - Se obtiene el inverso de mod respecto a p usando la función `inversa_mod_p`.
 - El valor de r se actualiza aplicando la fórmula basada en el CRT:

$$r = (r + (b - a \times r) \times m_{\text{inverso}} \times \text{mod}) \% (\text{mod} \times p)$$

- El módulo acumulado se actualiza multiplicándolo por p.
3. **Retorno:** La función devuelve el valor final de r y el módulo acumulado, que representan la solución general del sistema de congruencias.

La clave está en asegurar que los módulos sean coprimos, y en cada paso, la función calcula el inverso de m módulo p para encontrar el valor correcto de r que satisface todas las congruencias hasta ese punto. Utilizando como se ha visto en clase y se menciona anteriormente el Teorema Chino del Resto

1. **Uso del Inverso Modular:** La función calcula el inverso de m módulo p solo cuando es necesario, minimizando las operaciones y asegurando que los cálculos son eficientes.
2. **Cálculo iterativo y modular:** La función actualiza el valor de r y el módulo en cada iteración, lo que permite construir la solución de manera incremental y eficiente.
3. **Multiplicación acumulada de módulos:** Al multiplicar el módulo acumulado por p en cada iteración, la función asegura que el espacio de búsqueda de soluciones se incrementa adecuadamente para manejar cada congruencia.

Para realizar la función “run commands” hemos utilizado el lenguaje regex para identificar cada una de las funciones pedidas en los ficheros a leer. Hemos tenido que hacer uso de la librería re para importar el lenguaje regex que nos ha dado algún problema para la función de resolverSistema pero al final con ayuda hemos conseguido que funcionara correctamente. Luego hemos escrito el resultado o la excepción que saltaba en el fichero de salida. Hemos ido comprobando las funciones una a una para que entrara en la indicada y que no saltaran errores no deseados.

Luego el menú interactivo ha sido programado para que se introduzcan las funciones o los ficheros por máquina, empleando la librería sys para poder acceder a los elementos introducidos por la terminal. Hemos ido comprobando la longitud de los parámetros escritos por máquina para que no saltara la excepción IndexError por lo que hemos tenido que identificar las funciones según el número de parámetros que necesitaran.

Anexo

Teorema de referencia 1

El teorema utilizado para justificar la validez del algoritmo es:

Teorema: *Si un número compuesto n tiene un divisor, entonces tiene al menos un divisor menor o igual a $\sqrt{n}$.*

Esto se puede derivar de la propiedad de los factores: si $n=a \times b$ y ambos son mayores que $\sqrt{n}$, entonces el producto superaría n . Por lo tanto, para encontrar si n es primo, solo es necesario comprobar divisibilidad hasta $\sqrt{n}$.

Para una referencia formal, se puede consultar el **Teorema Fundamental de la Aritmética**, que asegura que cualquier número compuesto puede descomponerse en factores primos menores o iguales a su raíz cuadrada.

Bibliografía

<https://chatgpt.com>

https://es.wikipedia.org/wiki/Criba_de_Erat%C3%B3stenes

https://es.wikipedia.org/wiki/Teorema_fundamental_de_la_aritm%C3%A9tica#:~:text=El%20teorema%20fundamental%20implica%20que,un%20producto%20de%20n%C3%BAmeros%20primos.

Pdf (Teoría de Números *Departamento de Matemática Aplicada de ICAI*)

https://es.wikipedia.org/wiki/Exponenciación_modular

https://es.wikipedia.org/wiki/Teorema_chino_del_resto

